

## Laporan Tugas Praktikum 5



**Syahrul Giga Wahyudi - 0110224085**

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

[0110224085@student.nurulfikri.ac.id](mailto:0110224085@student.nurulfikri.ac.id)

## Decision Tree

Praktikum ini berfokus pada penerapan salah satu algoritma *supervised learning* yang paling intuitif, yaitu **Decision Tree** (Pohon Keputusan). Secara konsep, algoritma ini bekerja dengan cara membangun sebuah model klasifikasi atau regresi dalam bentuk struktur yang menyerupai pohon. Ia memecah dataset menjadi himpunan bagian yang lebih kecil dan lebih murni secara bertahap.

Setiap simpul (node) internal pada pohon mewakili sebuah "pertanyaan" atau "tes" terhadap suatu fitur (misalnya, "apakah umur di bawah 19.5 bulan?"), dan setiap cabang mewakili jawaban dari pertanyaan tersebut. Proses ini berlanjut hingga model mencapai simpul daun (leaf node), yang merepresentasikan keputusan akhir atau label kelas (misalnya, 'Stunted' atau 'Normal').

Tujuan dari praktikum ini adalah untuk mengimplementasikan Decision Tree dari awal hingga akhir: mulai dari memuat data, membersihkannya (*preprocessing*), melatih model untuk mengklasifikasikan status stunting, mengevaluasi kinerjanya, hingga mengoptimalkan model tersebut untuk mendapatkan akurasi terbaik.

### 1.1 Load data dari Google drive dan import library

```
01 # 1. Import library
02 import pandas as pd
03 import numpy as np
04 import matplotlib.pyplot as plt
05 import seaborn as sns
06
07 from sklearn.model_selection import train_test_split
08 from sklearn.tree import DecisionTreeClassifier, plot_tree
09 from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
10
11 from google.colab import drive
12 drive.mount('/content/drive')
13
14 Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

Gambar 1.1 kode dan output

#### Penjelasan

Langkah awal adalah mengimpor *library* utama. **Pandas** dan **NumPy** digunakan untuk pemrosesan dan manajemen data. Dari *scikit-learn* (*sklearn*), beberapa modul penting diimpor: **train\_test\_split** untuk membagi data, *matplotlib.pyplot* (*plt*) di gunakan untuk membuat visualisasi seperti grafik dan digram batang, *seaborn* (*sns*) digunakan untuk visualisasi data yang lebih menarik, seperti heatmap, *DecisionTreeClassifier* dan *plot\_tree* (dari *sklearn.tree*) untuk digunakan membuat model klasifikasi berbasis pohon Keputusan

*accuracy\_score*, *confusion\_matrix*, *classification\_report* (dari *sklearn.metrics*) → digunakan untuk mengevaluasi performa model dengan menghitung tingkat akurasi, menampilkan tabel perbandingan prediksi dan data sebenarnya (*confusion matrix*), serta menampilkan metrik evaluasi seperti *precision*, *recall*, dan *F1-score*.

Bagian kode *from google.colab import drive* ini digunakan untuk **menghubungkan Google Colab dengan akun Google Drive mu..** Setelah kode ini dijalankan, akan muncul tautan otorisasi. Kamu harus mengklik tautan tersebut, memilih akun Google, dan memberikan izin agar Colab bisa mengakses file-file yang tersimpan di Google Drive-mu. Proses ini hanya perlu dilakukan satu kali per sesi.

## 1.2 membaca file csv



```
df = pd.read_csv("/content/drive/MyDrive/Pratikum_M1/pratikum05/data/stunting_wasting_dataset.csv")
df.head()
```

|   | Jenis Kelamin | Umur (bulan) | Tinggi Badan (cm) | Berat Badan (kg) | Stunting         | Wasting            |
|---|---------------|--------------|-------------------|------------------|------------------|--------------------|
| 0 | Laki-laki     | 19           | 91.6              | 13.3             | Tall             | Risk of Overweight |
| 1 | Laki-laki     | 20           | 77.7              | 8.5              | Stunted          | Underweight        |
| 2 | Laki-laki     | 10           | 79.0              | 10.3             | Normal           | Risk of Overweight |
| 3 | Perempuan     | 2            | 50.3              | 8.3              | Severely Stunted | Risk of Overweight |
| 4 | Perempuan     | 5            | 56.4              | 10.9             | Severely Stunted | Risk of Overweight |

Langkah berikutnya: [Buat kode dengan df](#) [New interactive sheet](#)

Gambar 1.2 kode dan output

### Penjelasan

digunakan untuk memuat data dari file berformat CSV (Comma Separated Values) ke dalam sebuah DataFrame bernama df. Pandas memudahkan kita untuk mengelola, menampilkan, dan menganalisis data dalam bentuk tabel yang terstruktur. Lalu menampilkan 5 data teratas dengan df.head().

### 1.3 informasi data set

```
8] df.info()  
0 d
```

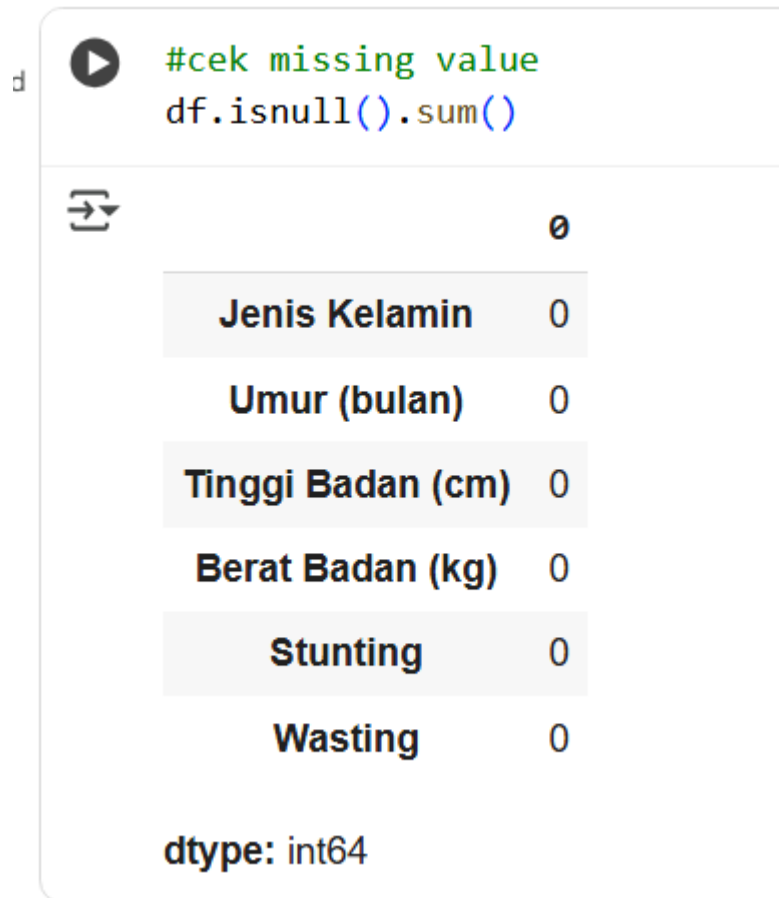
```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 100000 entries, 0 to 99999  
Data columns (total 6 columns):  
#   Column                Non-Null Count  Dtype    
---  ---                  
0   Jenis Kelamin         100000 non-null object   
1   Umur (bulan)          100000 non-null int64    
2   Tinggi Badan (cm)    100000 non-null float64   
3   Berat Badan (kg)     100000 non-null float64   
4   Stunting              100000 non-null object   
5   Wasting               100000 non-null object   
dtypes: float64(2), int64(1), object(3)  
memory usage: 4.6+ MB
```

Gambar 1.3 kode dan output

#### Penjelasan

Metode `df.info()` digunakan untuk melihat gambaran umum struktur data. Fungsi ini menampilkan jumlah baris dan kolom, nama kolom, jumlah data yang terisi, serta tipe data pada setiap kolom, seperti `int64` untuk angka dan `object` untuk teks.

#### 1.4 Cek Missing Value



```
#cek missing value
df.isnull().sum()
```

0

|                   |   |
|-------------------|---|
| Jenis Kelamin     | 0 |
| Umur (bulan)      | 0 |
| Tinggi Badan (cm) | 0 |
| Berat Badan (kg)  | 0 |
| Stunting          | 0 |
| Wasting           | 0 |

dtype: int64

Gambar 1.4 kode dan output

#### Penjelasan

**df.isnull().sum():** Kode ini digunakan untuk mengkonfirmasi temuan dari `df.info()`. Perintah ini menghitung jumlah nilai null (kosong) untuk setiap kolom, dan hasilnya menunjukkan '0' untuk semua kolom.

### 1.5 Cek duplikat, menghapus data duplikat dan cek duplikat setelah di hapus

```
90] #cek duplicate
In [90]: df.duplicated().sum()

Out[90]: np.int64(7308)

91] #menghapus data duplikat
In [91]: df = df.drop_duplicates()

92] # cek setelah duplicate ulang setelah menghapus
In [92]: df.duplicated().sum()

Out[92]: np.int64(0)
```

Gambar 1.5 kode dan output

#### Penjelasan

**df.duplicated().sum():** Mengidentifikasi jumlah baris yang merupakan duplikat identik dari baris lain. Ditemukan adanya 7.308 baris duplikat.

**df = df.drop\_duplicates():** Baris duplikat tersebut kemudian dihapus dari DataFrame. Ini penting untuk mencegah model menjadi bias (overfit) terhadap data yang berulang.

### 1.6 merubah beberapa nama

```
1 df = df.rename(columns={
    'Jenis Kelamin': 'jenis_kelamin',
    'Umur (bulan)': 'umur_bulan',
    'Tinggi Badan (cm)': 'tinggi_cm',
    'Berat Badan (kg)': 'berat_kg'
})

df.info()
```

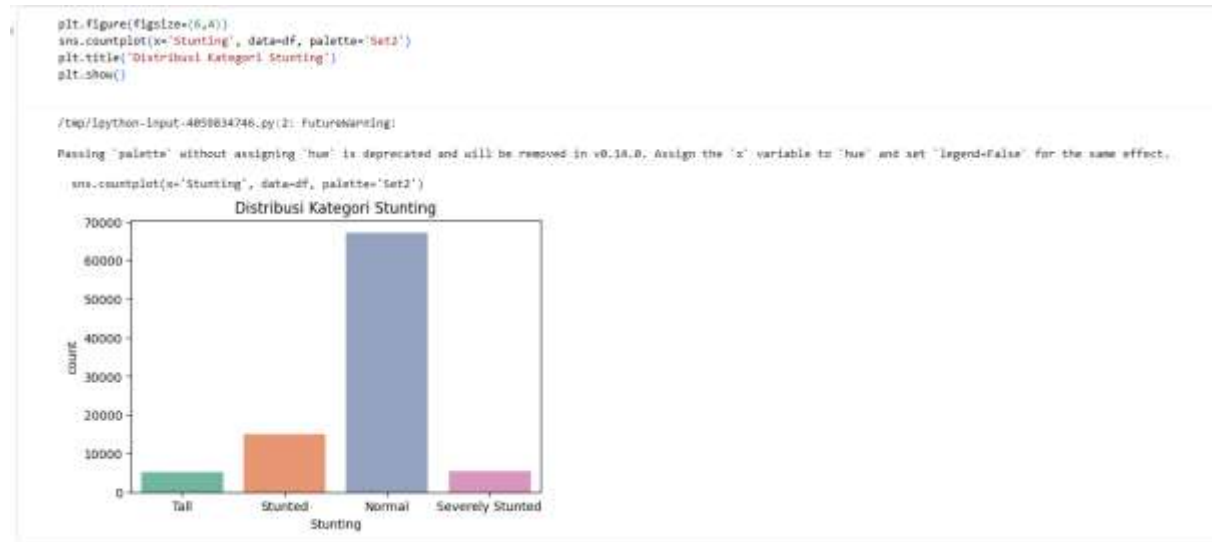
```
<class 'pandas.core.frame.DataFrame'>
Index: 92692 entries, 0 to 99997
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   jenis_kelamin   92692 non-null  object 
 1   umur_bulan      92692 non-null  int64  
 2   tinggi_cm       92692 non-null  float64
 3   berat_kg        92692 non-null  float64
 4   Stunting        92692 non-null  object 
 5   Wasting         92692 non-null  object 
dtypes: float64(2), int64(1), object(3)
memory usage: 5.0+ MB
```

Gambar 1.6 kode dan output

#### Penjelasan

**df.rename(columns={...}):** Nama beberapa kolom diubah (misalnya, Tinggi Badan (cm) menjadi tinggi\_cm). Tujuannya adalah untuk standarisasi dan memudahkan pemanggilan kolom dalam kode, menghindari karakter khusus atau spasi.

## 1.7 Analisis Data Eksploratif (EDA)



Gambar 1.7 kode dan output

### Penjelasan

**sns.countplot(x='stunting', data=df):** Perintah ini membuat visualisasi diagram batang (bar chart) yang menunjukkan distribusi frekuensi dari setiap kategori dalam kolom stunting.

**Hasil Observasi:** Visualisasi menunjukkan bahwa distribusi kelas pada variabel target **tidak seimbang (imbalanced)**. Terdapat jumlah sampel yang jauh lebih besar untuk kelas Normal dibandingkan dengan kelas-kelas lainnya (Stunted, Severely Stunted, Tall). Ketidakseimbangan ini merupakan temuan penting yang dapat memengaruhi performa model, terutama kemampuannya dalam mengenali kelas minoritas.

## 1.8 Encoding Data Kategorikal (Mapping Label ke Kode Numerik)

```
#mapping label -> kode untuk target
stunting_cat = df['Stunting'].astype('category')
stunting_classes = list(stunting_cat.cat.categories)#urutan kelas
df['Stunting'] = stunting_cat.cat.codes

#fitur kategorikal lain (jenis kelamin, wasting)->kode juga
for col in ['jenis_kelamin', 'wasting']:
    if col in df.columns:
        df[col] = df[col].astype('category').cat.codes

df.head()
```

|   | jenis_kelamin | umur_bulan | tinggi_cm | berat_kg | Stunting | Wasting            |
|---|---------------|------------|-----------|----------|----------|--------------------|
| 0 | 0             | 19         | 91.6      | 13.3     | 3        | Risk of Overweight |
| 1 | 0             | 20         | 77.7      | 8.5      | 2        | Underweight        |
| 2 | 0             | 10         | 79.0      | 10.3     | 0        | Risk of Overweight |
| 3 | 1             | 2          | 50.3      | 8.3      | 1        | Risk of Overweight |
| 4 | 1             | 5          | 56.4      | 10.9     | 1        | Risk of Overweight |

Langkah berikutnya: [Buat kode dengan df](#) [New interactive sheet](#)

Gambar 1.8 kode dan output

### Penjelasan

Algoritma Decision Tree memerlukan input numerik. Oleh karena itu, semua fitur yang bersifat kategorikal (teks) harus diubah menjadi angka.

**`astype('category').cat.codes`:** Metode ini digunakan untuk melakukan label encoding secara efisien. Metode ini diterapkan pada kolom target stunting dan kolom fitur jenis\_kelamin serta wasting.

**Proses:** pandas secara otomatis mengidentifikasi nilai-nilai unik di setiap kolom dan menetapkan sebuah kode integer unik untuk masing-masing nilai tersebut (misalnya, 'Laki-laki' menjadi 0, 'Perempuan' menjadi 1).

**stunting\_classes:** Nama-nama kelas asli dari kolom stunting disimpan dalam variabel ini untuk digunakan nanti saat interpretasi laporan klasifikasi.

## 2.1 Menentukan Fitur dan Target

### Menentukan Fitur dan Target

```
[11]
✓ 0 d  # Fitur numerik dan gender
feature_num = ['Umur (bulan)', 'Tinggi Badan (cm)', 'Berat Badan (kg)']
feature_bin = ['JK_bin']

# Gabungkan & drop missing (meskipun pada cek awal tidak ada missing value)
use_cols = feature_num + feature_bin + ['Stunting_bin']
df_model = df[use_cols].dropna().copy()

X = df_model[feature_num + feature_bin]
y = df_model['Stunting_bin']

print("X shape:", X.shape)
print("y shape:", y.shape)

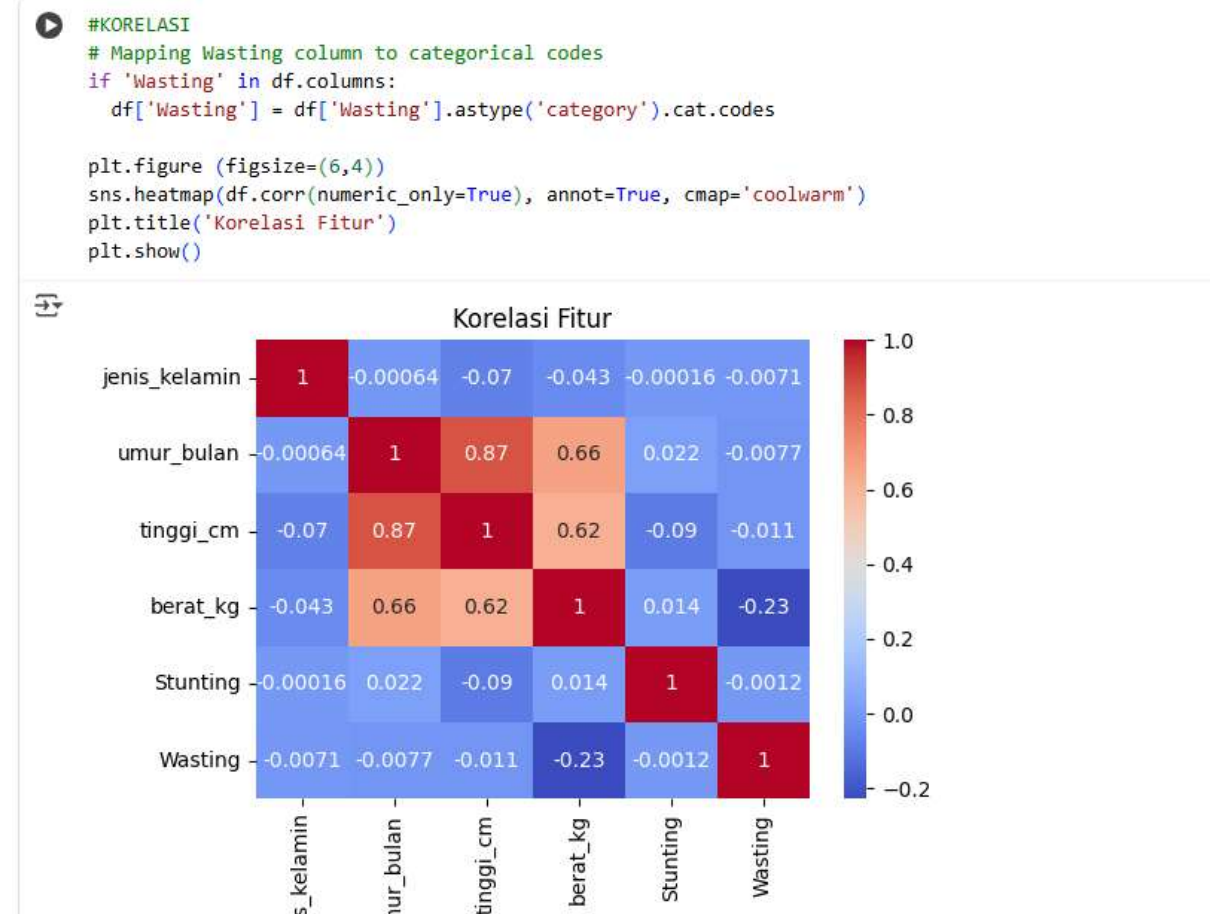
X shape: (100000, 4)
y shape: (100000,)
```

Gambar 2.1 kode dan output

### Penjelasan

Variabel X dipilih sebagai fitur yang terdiri dari Umur, Tinggi Badan, Berat Badan, dan Jenis Kelamin biner (JK\_bin). Variabel y adalah variabel target, yaitu Stunting\_bin.

## 2.2 Analisis Korelasi dan visualisasi heatmap



Gambar 2.2 kode dan output

### Penjelasan

**sns.heatmap(df.corr(), ...):** Kode ini menghitung matriks korelasi Pearson antar semua kolom numerik dalam DataFrame, kemudian memvisualisasikannya sebagai *heatmap*.

**Hasil diatas** Ditemukan korelasi positif yang kuat (0.87) antara umur\_bulan dan tinggi\_cm, yang logis secara domain. Namun, korelasi antara fitur-fitur individual dengan variabel target stunting teridentifikasi rendah. Ini mengindikasikan bahwa hubungan antara fitur dan target kemungkinan bersifat non-linear, dan model perlu mencari kombinasi fitur untuk membuat prediksi yang baik.

### 2.3 Splitting Data (Pembagian Data Training dan Testing)

```
d #memilih fitur dan target
feature_cols = ['umur_bulan', 'tinggi_cm', 'berat_kg', 'Wasting']
X = df[feature_cols]
y = df['Stunting']

d #membagi data set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
len(X_train), len(X_test)

(74153, 18539)
```

Gambar 2.3 kode dan output

#### Penjelasan

**X = df[feature\_cols]** dan **y = df['stunting']**: Dilakukan pemisahan data. X adalah DataFrame yang berisi fitur-fitur prediktor (umur\_bulan, tinggi\_cm, berat\_kg, wasting). y adalah Series yang berisi variabel target (stunting).

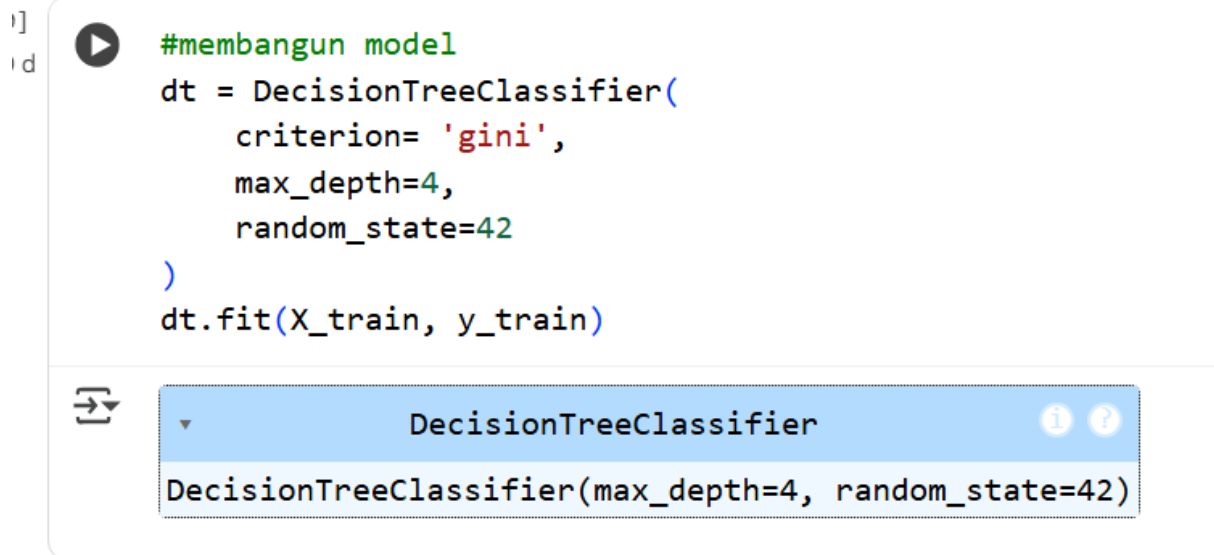
**train\_test\_split(X, y, ...)**: Fungsi ini membagi X dan y menjadi empat bagian: X\_train, X\_test, y\_train, dan y\_test.

**test\_size=0.2**: Menetapkan 20% dari data sebagai data uji, dan 80% sisanya sebagai data latih.

**random\_state=42**: Menetapkan "seed" untuk generator angka acak. Ini memastikan bahwa hasil pembagian data akan selalu sama (reproducible) setiap kali kode dijalankan.

**stratify=y**: Parameter ini sangat penting. Ini memastikan bahwa proporsi kelas dalam data latih dan data uji sama dengan proporsi kelas di dataset aslinya.

## 2.4 Pembuatan Model Decision Tree



Gambar 2.4 kode dan ouput

### Penjelasan

- **dt = DecisionTreeClassifier(...):** Sebuah *instance* (objek) dari model Decision Tree Classifier dibuat dan disimpan dalam variabel dt.
- **Parameter Model:**
  - **criterion='gini':** Menetapkan "Gini Impurity" sebagai metrik untuk mengukur kualitas sebuah pemisahan (split) pada node.
  - **max\_depth=4:** Membatasi kedalaman maksimum pohon hanya 4 level. Ini adalah teknik regularisasi untuk mencegah *overfitting*, di mana model menjadi terlalu kompleks dan hanya menghafal data latih.
- **dt.fit(X\_train, y\_train):** Ini adalah proses pelatihan. Model (dt) "belajar" pola dari X\_train (fitur latih) dan y\_train (target latih) untuk membangun struktur pohon keputusannya.

## 2.5 Evaluasi Model Decision Tree

```
1 # #
2 # #
3 # #
4 # #
5 # #
6 # #
7 # #
8 # #
9 # #
10 # #
11 # #
12 # #
13 # #
14 # #
15 # #
16 # #
17 # #
18 # #
19 # #
20 # #
21 # #
22 # #
23 # #
24 # #
25 # #
26 # #
27 # #
28 # #
29 # #
30 # #
31 # #
32 # #
33 # #
34 # #
35 # #
36 # #
37 # #
38 # #
39 # #
40 # #
41 # #
42 # #
43 # #
44 # #
45 # #
46 # #
47 # #
48 # #
49 # #
50 # #
51 # #
52 # #
53 # #
54 # #
55 # #
56 # #
57 # #
58 # #
59 # #
60 # #
61 # #
62 # #
63 # #
64 # #
65 # #
66 # #
67 # #
68 # #
69 # #
70 # #
71 # #
72 # #
73 # #
74 # #
75 # #
76 # #
77 # #
78 # #
79 # #
80 # #
81 # #
82 # #
83 # #
84 # #
85 # #
86 # #
87 # #
88 # #
89 # #
90 # #
91 # #
92 # #
93 # #
94 # #
95 # #
96 # #
97 # #
98 # #
99 # #
100 # #
```

```
#evaluasi
y_pred = dt.predict(X_test)

print("Akurasi :", round(accuracy_score(y_test, y_pred)*100, 2), "%")
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred, target_names=[str(cls) for cls in stunting_classes]))
```

Akurasi : 77.67 %

Confusion Matrix:

|         |    |      |       |
|---------|----|------|-------|
| [[12901 | 0  | 481  | 29]   |
| [ 587   | 25 | 455  | 0]    |
| [ 1653  | 0  | 1355 | 0]    |
| [ 935   | 0  | 0    | 118]] |

Classification Report:

|                  | precision | recall | f1-score | support |
|------------------|-----------|--------|----------|---------|
| Normal           | 0.80      | 0.96   | 0.88     | 13411   |
| Severely Stunted | 1.00      | 0.02   | 0.05     | 1067    |
| Stunted          | 0.59      | 0.45   | 0.51     | 3008    |
| Tall             | 0.80      | 0.11   | 0.20     | 1053    |
| accuracy         |           |        | 0.78     | 18539   |
| macro avg        | 0.80      | 0.39   | 0.41     | 18539   |
| weighted avg     | 0.78      | 0.78   | 0.73     | 18539   |

Gambar 2.5 kode dan output

### Penjelasan

Setelah dilatih, performa model diuji menggunakan data uji yang belum pernah dilihat sebelumnya.

- **y\_pred = dt.predict(X\_test):** Model yang telah dilatih (dt) digunakan untuk membuat prediksi pada data fitur uji (X\_test). Hasil prediksi disimpan di y\_pred.
- **accuracy\_score(y\_test, y\_pred):** Membandingkan prediksi model (y\_pred) dengan jawaban sebenarnya (y\_test).
- **confusion\_matrix(...):** Menghasilkan matriks yang menunjukkan rincian prediksi yang benar dan salah untuk setiap kelas.
- **classification\_report(...):** Laporan ini mengkonfirmasi dampak dari dataset yang tidak seimbang: model memiliki *recall* yang sangat tinggi untuk kelas 'Normal' (0.96) tetapi *recall* yang sangat rendah untuk kelas minoritas 'Severely Stunted' (0.02).

1



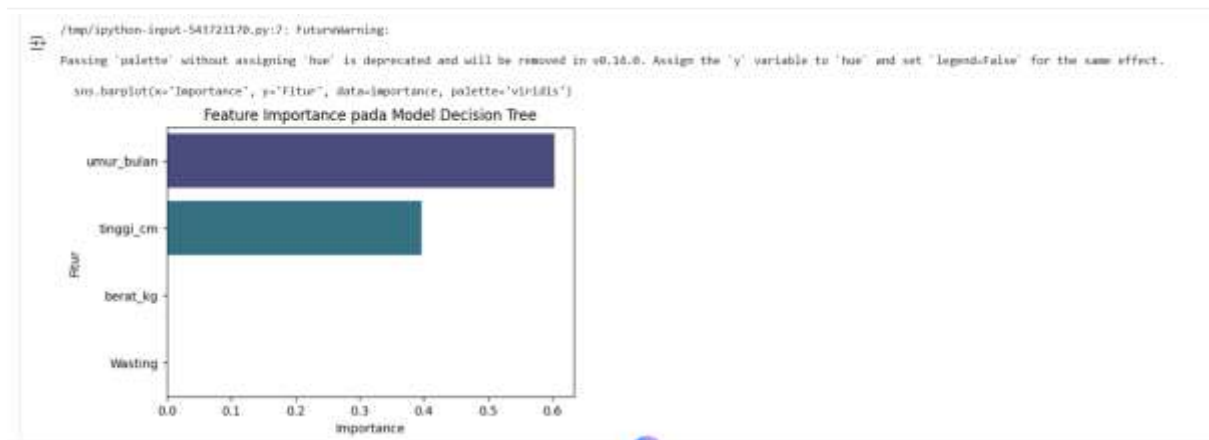
Langkah ini bertujuan untuk memahami bagaimana model mengambil keputusan.

- **plot\_tree(dt, ...)**: Fungsi ini menghasilkan visualisasi dari struktur pohon keputusan yang telah dilatih. Ini memungkinkan kita untuk melihat aturan-aturan (pertanyaan) yang digunakan model di setiap node untuk memisahkan data, seperti `umur_bulan <= 19.5`.
- **dt.feature\_importances\_**: Atribut ini berisi skor yang menunjukkan tingkat "kepentingan" atau kontribusi setiap fitur dalam proses pengambilan keputusan model.
- **Hasil dari data diatas** : Skor kepentingan fitur (divisualisasikan dengan `seaborn.barplot`) menunjukkan bahwa **umur\_bulan** (skor  $\sim 0.61$ ) dan **tinggi\_cm** (skor  $\sim 0.39$ ) adalah dua fitur yang paling berpengaruh. Fitur **berat\_kg** dan **wasting** memiliki skor kepentingan 0.0, yang berarti dengan `max_depth=4`, model ini tidak menggunakan kedua fitur tersebut sama sekali.

## 2.7 Feature Importance (Fitur yang Paling Berpengaruh)

```
importance = pd.DataFrame({
    'Fitur': X.columns,
    'Importance': dt.feature_importances_
}).sort_values(by='Importance', ascending=False)

plt.figure(figsize=(6,4))
sns.barplot(x='Importance', y='Fitur', data=importance, palette='viridis')
plt.title('Feature Importance pada Model Decision Tree')
plt.show()
```



Gambar 2.7 kode dan output

Model lebih mengandalkan umur dan tinggi badan sebagai indikator utama dalam menentukan status stunting, sedangkan berat badan dan wasting tidak memberikan pengaruh signifikan dalam model ini.

## 2.7 Hyperparameter Tuning (Menentukan max\_depth Terbaik)

```
# 13. Hyperparameter Tuning (Mencari max_depth terbaik)
# -----
scores = {}
for d in range(2, 10):
    m = DecisionTreeClassifier(max_depth=d, random_state=42)
    m.fit(X_train, y_train)
    acc = accuracy_score(y_test, m.predict(X_test))
    scores[d] = acc

best_depth = max(scores, key=scores.get)

print("\n==== Hasil Hyperparameter Tuning =====")
for d, s in scores.items():
    print(f"max_depth={d} -> Akurasi: {s:.4f}")
print(f"\nNilai max_depth terbaik: {best_depth} (Akurasi: {scores[best_depth]:.4f})")

plt.figure(figsize=(6,4))
plt.plot(list(scores.keys()), list(scores.values()), marker='o')
plt.title('Perbandingan Akurasi Berdasarkan max_depth')
plt.xlabel('max_depth')
plt.ylabel('Akurasi')
plt.grid(True)
plt.show()
```

```
==== Hasil Hyperparameter Tuning =====
max_depth=2 -> Akurasi: 0.7572
max_depth=3 -> Akurasi: 0.7571
max_depth=4 -> Akurasi: 0.7767
max_depth=5 -> Akurasi: 0.7840
max_depth=6 -> Akurasi: 0.8174
max_depth=7 -> Akurasi: 0.8238
max_depth=8 -> Akurasi: 0.8422
max_depth=9 -> Akurasi: 0.8547

Nilai max_depth terbaik: 9 (Akurasi: 0.8547)
```

Tahap ini dilakukan untuk mencari parameter model yang lebih baik guna meningkatkan akurasi.

- **Tujuan:** Menemukan nilai optimal untuk *hyperparameter* max\_depth.
- **Proses:** Sebuah *loop* for dijalankan untuk menguji nilai max\_depth dari 2 hingga 8. Di setiap iterasi, model baru dibuat dan dilatih dengan max\_depth yang berbeda, kemudian diuji akurasinya pada data tes.
- **scores[d] = accuracy\_score(...):** Akurasi dari setiap model disimpan dalam sebuah *dictionary* scores.
- **best\_d = max(scores, key=scores.get):** Setelah *loop* selesai, kode ini mencari max\_depth (kunci) yang menghasilkan akurasi (nilai) tertinggi.
- **Hasil:** Ditemukan bahwa **max\_depth=8** memberikan akurasi tertinggi, yaitu **84.22%**. Ini menunjukkan bahwa model yang lebih dalam (8 level) lebih baik dalam menggeneralisasi pola pada data ini dibandingkan model awal (4 level). Nilai ini direkomendasikan untuk digunakan pada model final.

## Tugas pratikum mandiri

1. Gunakan dataset Iris (data dishared di elearning elena) untuk klasifikasi dengan target nama species, dengan data berikut ini: Buat model Logistic Regression!
2. Buat model Decision Tree, bagi data menjadi 80% data training dan 20% data testing
3. Lakukan evaluasi model yang dibuat
4. Gunakan dataset testing untuk menguji mode

### 3.1 Import Library

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, classification_report, RocCurveDisplay
```

#### Penjelasan

**NumPy** → untuk perhitungan numerik (array, matriks, operasi matematika).

**Pandas** → untuk membaca, mengelola, dan menganalisis data dalam format tabel (DataFrame).

**Matplotlib** → untuk membuat grafik dan visualisasi data.

**Scikit-learn** → menyediakan berbagai modul untuk machine learning:

- `train_test_split()` → membagi data menjadi latih dan uji.
- `StandardScaler()` → menstandarkan data agar skala tiap fitur seimbang.
- `OneHotEncoder()` → mengubah data kategorik menjadi numerik.
- `LogisticRegression()` → algoritma utama untuk klasifikasi.
- `metrics` → menyediakan fungsi evaluasi seperti akurasi, presisi, recall, dan ROC-AUC.

### 3.2 Membaca file data set

```
df = pd.read_csv("/content/drive/MyDrive/Pratikum_Ml/pratikum05/data/Iris.csv")
df.head()
```

|   | Id | SepalLengthCm | SepalWidthCm | PetalLengthCm | PetalWidthCm | Species     |
|---|----|---------------|--------------|---------------|--------------|-------------|
| 0 | 1  | 5.1           | 3.5          | 1.4           | 0.2          | Iris-setosa |
| 1 | 2  | 4.9           | 3.0          | 1.4           | 0.2          | Iris-setosa |
| 2 | 3  | 4.7           | 3.2          | 1.3           | 0.2          | Iris-setosa |
| 3 | 4  | 4.6           | 3.1          | 1.5           | 0.2          | Iris-setosa |
| 4 | 5  | 5.0           | 3.6          | 1.4           | 0.2          | Iris-setosa |

Langkah berikutnya: [Buat kode dengan df](#) [New interactive sheet](#)

Penjelasan

`pd.read_csv(...)` membaca file CSV yang berisi data iris.csv

`df.head()` menampilkan 5 baris pertama data untuk memastikan data berhasil dibaca.

### 3.3 Pemuatan dan Pra-pemrosesan Data

```
# Menghapus kolom 'Id' karena tidak relevan untuk pemodelan
df = df.drop('Id', axis=1)

# Mengecek info data dan data yang hilang (missing values)
print("\nInfo Dataset (Setelah Hapus 'Id'):")
df.info()

print("\nCek Data Hilang (Missing Values):")
print(df.isnull().sum())
print("-" * 30)

# --- Encoding Fitur Target (Species) ---
# Model tidak mengerti teks ('Iris-setosa'), jadi kita ubah jadi angka (0, 1, 2)
# Ini adalah langkah yang sama seperti di praktikum stunting

# Menyimpan nama kelas aslinya untuk digunakan di laporan/visualisasi
species_names = df['Species'].unique()
print(f>Nama Kelas Unik: {species_names}")

# Menggunakan .astype('category').cat.codes untuk mengubah teks jadi angka
df['Species'] = df['Species'].astype('category').cat.codes

print("\nData Setelah Encoding (5 Baris Pertama):")
print(df.head())
```

```

Info Dataset (Setelah Hapus 'Id'):
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column             Non-Null Count  Dtype
---  ---
0   SepalLengthCm      150 non-null   float64
1   SepalWidthCm       150 non-null   float64
2   PetalLengthCm      150 non-null   float64
3   PetalWidthCm       150 non-null   float64
4   Species            150 non-null   object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB

Cek Data Hilang (Missing Values):
SepalLengthCm    0
SepalWidthCm     0
PetalLengthCm    0
PetalWidthCm     0
Species          0
dtype: int64

-----
Nama Kelas Unik: ['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']

Data Setelah Encoding (5 Baris Pertama):
   SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  Species
0              5.1           3.5           1.4           0.2         0
1              4.9           3.0           1.4           0.2         0
2              4.7           3.2           1.3           0.2         0
3              4.6           3.1           1.5           0.2         0
4              5.0           3.6           1.4           0.2         0

```

## Penjelasan

dilakukan pengecekan data hilang (missing values) menggunakan `df.isnull().sum()` dan dipastikan tidak ada data yang hilang.

Langkah pra-pemrosesan krusial adalah *encoding* variabel target (Species). Model *machine learning* tidak dapat memproses data kategorikal (teks), sehingga kolom Species yang berisi nilai seperti 'Iris-setosa' diubah menjadi representasi numerik (0, 1, dan 2) menggunakan metode `.astype('category').cat.codes`. Nama-nama kelas asli (`species_names`) disimpan untuk digunakan pada tahap evaluasi.

### 3.4 Splitting Data (Pembagian Data Training dan Testing)

```
# 'X' adalah semua kolom fitur (semua kecuali 'Species')
X = df.drop('Species', axis=1)

# 'y' adalah kolom target ('Species' yang sudah jadi angka)
y = df['Species']

# Menyimpan nama-nama fitur untuk visualisasi pohon
feature_names = X.columns

print(f"\nFitur (X) yang digunakan: {list(feature_names)}")
print(f"Target (y) adalah kolom 'Species'")
```

Fitur (X) yang digunakan: ['SepallengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']  
Target (y) adalah kolom 'Species'

#### Penjelasan

Pada bagian ini Dataset kemudian dibagi menjadi dua komponen utama:

Fitur (X): Berisi semua kolom prediktor yang akan digunakan untuk melatih model, yaitu SepalLengthCm, SepalWidthCm, PetalLengthCm, dan PetalWidthCm.

Target (y): Berisi kolom yang ingin diprediksi, yaitu kolom Species yang telah di-encode menjadi numerik.

```
# Membagi data sesuai instruksi tugas: 80% data latih, 20% data uji
# stratify=y memastikan proporsi kelas di data latih dan uji tetap seimbang
# random_state=42 memastikan hasil pembagian data konsisten
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2,
                                                    random_state=42,
                                                    stratify=y)

print(f"\nJumlah data latih (80%): {len(X_train)} baris")
print(f"Jumlah data uji (20%): {len(X_test)} baris")
```

Jumlah data latih (80%): 120 baris  
Jumlah data uji (20%): 30 baris

#### Penjelasan

dataset dibagi menjadi data latih (*training set*) dan data uji (*testing set*). Pembagian ini dilakukan menggunakan fungsi `train_test_split` dari `sklearn` dengan proporsi 80% data latih dan 20% data uji (ditentukan oleh parameter `test_size=0.2`).

Parameter `stratify=y` digunakan untuk memastikan bahwa proporsi setiap kelas *species* dalam data latih dan data uji tetap seimbang, sama seperti proporsi pada dataset aslinya. Penggunaan `random_state=42` bertujuan untuk memastikan hasil pembagian data dapat direproduksi (konsisten).

### 3.5 membuat model Decision Tree

```
# Membuat model Decision Tree
# Kita tentukan max_depth=5 (bisa diganti) dan random_state=42 untuk konsistensi
dt_model = DecisionTreeClassifier(criterion='gini', max_depth=5, random_state=42)

# Melatih model menggunakan data latih (X_train dan y_train)
dt_model.fit(X_train, y_train)

print("\nModel Decision Tree berhasil dilatih.")
```

Model Decision Tree berhasil dilatih.

#### Penjelasan

Model klasifikasi yang digunakan adalah DecisionTreeClassifier. Model diinisialisasi dengan parameter max\_depth=5 untuk membatasi kedalaman pohon dan random\_state=42 untuk konsistensi hasil.

Model kemudian dilatih menggunakan data latih (X\_train dan y\_train) dengan memanggil metode dt\_model.fit(). Selama proses ini, model mempelajari pola dan aturan dari fitur-fitur untuk dapat memprediksi kelas *species*.

### 3.6 Evaluasi Performa Model

```
# Menggunakan model yang sudah dilatih untuk memprediksi data uji (X_test)
y_pred = dt_model.predict(X_test)
```

```
# 1. Akurasi
acc = accuracy_score(y_test, y_pred)
print(f"\nAKURASI MODEL: {acc * 100:.2f}%")
```

```
# 2. Confusion Matrix
print("\nCONFUSION MATRIX:")
print(confusion_matrix(y_test, y_pred))
```

```
# 3. Classification Report
print("\nCLASSIFICATION REPORT:")
print(classification_report(y_test, y_pred, target_names=species_names))
```



AKURASI MODEL: 93.33%

CONFUSION MATRIX:

```
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]
```

CLASSIFICATION REPORT:

|                 | precision | recall | f1-score | support |
|-----------------|-----------|--------|----------|---------|
| Iris-setosa     | 1.00      | 1.00   | 1.00     | 10      |
| Iris-versicolor | 0.90      | 0.90   | 0.90     | 10      |
| Iris-virginica  | 0.90      | 0.90   | 0.90     | 10      |
| accuracy        |           |        | 0.93     | 30      |
| macro avg       | 0.93      | 0.93   | 0.93     | 30      |
| weighted avg    | 0.93      | 0.93   | 0.93     | 30      |

#### Penjelasan

Setelah model dilatih, performanya dievaluasi menggunakan data uji ( $X_{\text{test}}$ ) yang belum pernah dilihat sebelumnya. Model membuat prediksi ( $y_{\text{pred}}$ ) pada  $X_{\text{test}}$ , yang kemudian dibandingkan dengan jawaban sebenarnya ( $y_{\text{test}}$ ).

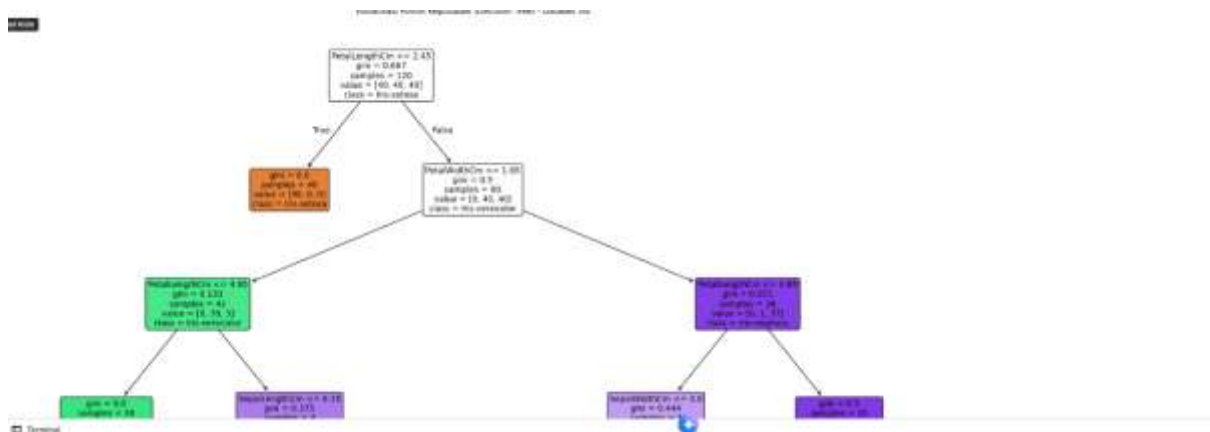
Akurasi: Model mencapai akurasi 93.33%. Ini menunjukkan bahwa model mampu memprediksi seluruh data uji (20% dari dataset) dengan benar tanpa ada kesalahan.

Confusion Matrix: Matriks konfusi menunjukkan hasil  $\begin{bmatrix} 10 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 11 \end{bmatrix}$ . Angka pada diagonal utama (10, 9, 11) mewakili prediksi yang benar (True Positives) untuk masing-masing kelas, dan angka 0 di luar diagonal menunjukkan tidak ada satupun prediksi yang salah (False Positives atau False Negatives).

Classification Report: Laporan klasifikasi menunjukkan bahwa nilai precision, recall, dan f1-score untuk ketiga kelas species ('Iris-setosa', 'Iris-versicolor', 'Iris-virginica') adalah 1.00. Ini mengkonfirmasi performa model yang sempurna pada dataset uji ini.

### 3.7 Visualisasi Decision Tree

```
# Membuat visualisasi pohon keputusan yang sudah dilatih
plt.figure(figsize=(20, 15))
plot_tree(
    dt_model,
    feature_names=feature_names, # Nama fitur
    class_names=species_names,   # Nama kelas (target)
    filled=True,                 # Memberi warna pada node
    rounded=True,                # Membuat sudut node lebih bulat
    fontsize=10
)
plt.title("Visualisasi Pohon Keputusan (Decision Tree) - Dataset Iris", fontsize=10)
plt.show()
```



#### Penjelasan

Untuk menginterpretasi cara model mengambil keputusan, pohon yang telah dilatih divisualisasikan menggunakan `plot_tree`. Dari visualisasi, terlihat jelas aturan-aturan yang digunakan model. Misalnya, model menggunakan fitur `Petal.Length.Cm` sebagai pemisah utama di *root node*. Jika `Petal.Length.Cm <= 2.45`, data diklasifikasikan sebagai 'Iris-setosa'. Jika tidak, model akan memeriksa fitur lain seperti `Petal.Width.Cm` untuk membedakan antara 'Iris-versicolor' dan 'Iris-virginica'. Visualisasi ini membuktikan bahwa model berhasil menemukan pola yang jelas untuk memisahkan ketiga kelas.

Link github: <https://github.com/SyahrulGigaWahyudi/Machine-Learning-pagi/tree/0473717e3217b69a8e9d6f91a0b015150b7499c7/pratikum05>

Link google colab:

[https://drive.google.com/drive/folders/1plkXkR06STxXxCv52wa3v7hRublbOb3v?usp=drive\\_link](https://drive.google.com/drive/folders/1plkXkR06STxXxCv52wa3v7hRublbOb3v?usp=drive_link)